
Desarrollo Web entorno-cliente

—— UT2 – Introducción al Lenguaje ——
JavaScript

Introducción

Bases de la Programación

Continuamos con la iniciación a JavaScript, repasando los fundamentos esenciales del desarrollo de software.

Trabajo en el Aula

Durante la unidad de trabajo realizaremos múltiples actividades prácticas in-situ para consolidar el aprendizaje directamente en clase.



Evaluación y Entregables

El progreso se evaluará mediante actividades entregables ponderadas y una prueba final escrita que medirá tus conocimientos teóricos y prácticos.

Contenidos

01

Normas Sintácticas

- Palabras reservadas

02

Variables y Alcance

Declaración, ciclo de vida y visibilidad de los datos.

03

Tipos de datos

Estructuras de información primitivas y complejas.

04

Operadores

Aritméticos, lógicos y operaciones de asignación.

05

Flujo de ejecución

Estructuras de control condicionales y bucles.

06

Funciones

Modularización, parámetros y retorno de valores.

Selección del Lenguaje

98%+ de la Web

JavaScript es el rey indiscutible de las aplicaciones cliente web.

Interactúa de forma inmediata con el navegador, el DOM y eventos de manera fluida y nativa.



1. Ejecución Nativa

Es el único lenguaje que ejecutan de forma directa y nativa todos los navegadores web modernos sin necesidad de plugins o intérpretes externos.



2. Carga del Servidor

Realiza validaciones y dinámicas en la máquina local del usuario, evitando transferencias de red innecesarias y acelerando la velocidad de respuesta.



3. El Estándar Web

Constituye el núcleo del desarrollo web moderno de cliente junto con HTML5 y CSS3, sirviendo de base para dominar frameworks como Angular, React o Vue.

Normas Sintácticas

Regla Fundamental

Sensible al uso de mayúsculas/minúsculas



Recomendaciones

- **Nombres en minúscula:** para funciones y objetos.
- **Usar lowerCamelCase:** estilo de escritura recomendado para identificadores.



Comentarios

- **Una línea:** `// comentario`
- **Varias líneas:** `/* comentario */`

Normas Sintácticas II

Estructura de un Programa

Un script de JavaScript es una secuencia de sentencias (instrucciones sobre datos).

Referencia: [Enlace](#)

<> Punto y Coma

Cada instrucción debe terminar con un punto y coma (;) para separar correctamente las sentencias.

⌞ Espacios en Blanco

Los espacios consecutivos son considerados como un único espacio. El compilador ignora el espaciado extra.

📖 Palabras Reservadas

Palabras clave especiales reservadas por el lenguaje que no pueden usarse como identificadores.

[Ver lista completa](#)

Variables

Concepto Clave

Zonas de la memoria que se identifican con un nombre y en las cuales se almacenan datos.



Desarrollo de un Script

- **Declaración de variables:**
var, let y const
(se desaconseja el uso de var en favor de let y const debido a los problemas de ámbito y buenas prácticas en el desarrollo)
- **Inicialización de variables:**
Asignación de valor.



Normas del Identificador

- **Caracteres permitidos:**
Letras, \$ (dólar) y _ (guión bajo).
- **Restricción de inicio:**
El primer carácter no puede ser un número.
El espacio no está permitido.

Actividad 1

Recurso de Consulta

Visita la siguiente URL para realizar la actividad: [w3schools.com/js/js_variables.asp](https://www.w3schools.com/js/js_variables.asp)

Pregunta 1

Diferencias de alcance y uso:

¿Para qué sirven **var**, **let** y **const**, y cuándo debemos utilizar cada una de ellas?

Pregunta 2

Análisis de sintaxis:

¿Qué tipo de declaración es la siguiente instrucción en JavaScript?

```
x = 3;
```

Alcance

Ámbito Global

- **Fuera de funciones:** Las variables declaradas fuera de cualquier función son accesibles en todo el código.
- **No declaradas:** Si se asigna valor a una variable no declarada, se considera global por defecto.

Ámbito Local

- **Dentro de funciones:** Las variables declaradas dentro de una función solo existen y son accesibles dentro de ella.
- **Conflicto de nombres:** Si una variable local tiene el mismo nombre que una global, se prioriza la local dentro de la función.

Alcance de Bloque (let & const)

Estas palabras clave introducen variables que solo viven dentro del bloque `{ ... }` donde fueron declaradas.

Más información: [Scope](#)

¿Qué es el alcance de bloque?

<> Ejemplo de Código

```
{  
  let variable = 5;  
  // Accesible aquí ✓  
}  
// No accesible fuera ✗
```

Las llaves { ... } delimitan el bloque de código.



Reglas de Alcance

¿Cómo funciona?

- **Existencia Limitada:** La variable se crea al entrar al bloque y se destruye al salir.
- **Inaccesibilidad:** Intentar acceder a ella fuera del bloque produce un error de referencia.
- **Soporte Moderno:** Se aplica estrictamente al declarar variables con **let** y **const**.

Actividad 2

<> Particularidades de let

Investiga sus reglas de declaración, reasignación y cómo se comporta respecto al alcance de bloque:

Recurso de estudio: [\[let\]](#)

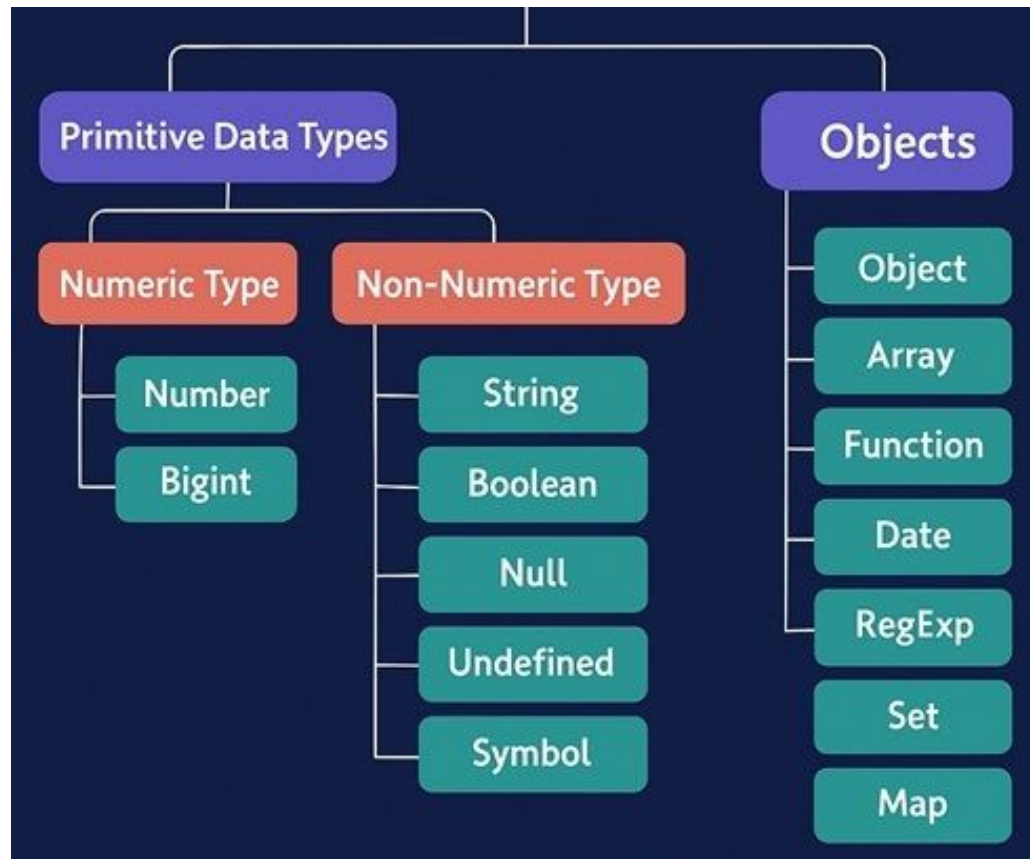
🔒 Particularidades de const

Analiza la inmutabilidad de la referencia, restricciones de inicialización y su comportamiento en bloques:

Recurso de estudio: [const](#)

Actividad de investigación: Compara la declaración, reasignación y el alcance (scope) de ambas palabras clave.

Tipos de datos



Tipos de datos: Number & String

Lenguaje débilmente tipado: el tipo de datos no es una propiedad de las variables, sino de los datos, siendo dinámico.

Recurso de estudio: [w3schools.com/js/js_datatypes.asp](https://www.w3schools.com/js/js_datatypes.asp)



Number

Representación y precisión de valores numéricos.

- Admite notación **decimal** (12.56), **hexadecimal** (0xA1F2) o **científica** (1256e-2).
- Valores comprendidos estrictamente entre **Number.MAX_VALUE** y **Number.MIN_VALUE**.
- **0.1 + 0.2 == 0.3** evalúa a **false** por falta de representación exacta en IEEE 754.

String

Cadenas de texto y secuencias de caracteres.

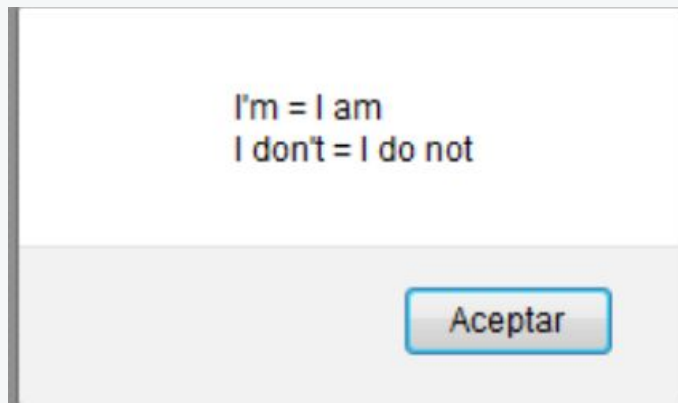
- Se delimita mediante **comillas simples** o **dobles** de manera indiferente.
- Soporta secuencias de escape como **\'**, **\"**, **** y otras 6 opciones.
- Representación bajo estándar **Unicode de 16 bits** usando la sintaxis **\unnnn**.

Actividad 3

Instrucciones de la Actividad

Crea un archivo HTML e incorpora el código JavaScript necesario para mostrar el siguiente texto en una ventana emergente.

Utiliza la función `alert()` una única vez.



El resultado obtenido debe ser el anterior

Conversión de Datos: Conversión Explícita

La Necesidad de la Conversión Explícita:

- **Entrada de usuario:** Los mecanismos de lectura como `prompt()` devuelven siempre cadenas de caracteres (Strings).
- **Riesgo de concatenación:** Si realizas sumas matemáticas directas, JS interpretará el `+` como concatenación (ej: `'5' + 5` resultará en `'55'`).
- **Buenas prácticas:** Es imperativo dominar y aplicar funciones para transformar tipos de datos antes de operarlos.

```
// Entrada del usuario: prompt()
let entrada = prompt("Introduce un número:"); // "5"

// Conversión explícita obligatoria:
let numero = Number(entrada); // 5 (Number)
let entero = parseInt(entrada); // 5
(Number entero)
let decimal = parseFloat(entrada); // 5
(Decimal)

// Otras conversiones explícitas:
let texto = String(100); // "100" (String)
let booleano = Boolean(1); // true
(Boolean)
```

Tipos de datos

Boolean

true y false

Object

Colección de datos, sin restricción en cuanto a tipo, identificados cada uno por un nombre.

También pueden contener **array** o **date**.

Undefined

Sólo admite el dato **undefined**.

Se aplica implícitamente a todas las variables declaradas pero no asignadas.

Symbol

Tipo de datos para crear identificadores únicos cuando se trabaja con objetos.

Null

Sólo admite el dato **null**.

Es la respuesta a un convenio de programadores que querían un tipo para variables que contendrán objetos pero aún no han sido asignadas.

No se puede aplicar implícitamente.

Actividad 4

Preguntas de Investigación

1. ¿Cuál es la diferencia entre el tipo BigInt y Number de JavaScript?

Investiga en:

https://www.w3schools.com/js/js_bigint.asp

2. Prueba el código de la derecha. ¿Cuál es el valor mostrado para cada variable? ¿Por qué?

```
let primer_saludo = "hola";
let segundo_saludo = primer_saludo;
primer_saludo = "hello";
alert(primer_saludo);
alert(segundo_saludo);

let colores = new Object();
colores.favorito = 'azul';
let miscolores = colores;
miscolores.favorito = 'rojo';
alert(colores.favorito);
```

Tipos de datos

Objetos del Core

JavaScript provee un conjunto de objetos predefinidos integrados nativamente en su core que facilitan el desarrollo:

- Number, String, Boolean (constructores de tipo)
- Array (manejo de listas indexadas)
- Function (constructor de funciones)
- Date (operaciones con fecha/hora)
- Math (operaciones matemáticas avanzadas)

Envolventes Temporales

Al invocar un método o propiedad sobre una variable primitiva (ej: un **String**), JavaScript la envuelve temporalmente en su correspondiente objeto envolvente (**String**) para dar acceso al método, y luego la destruye.

```
// El primitivo se envuelve al vuelo  
let clave = "juanfer";  
clave.length; // Accede a la propiedad  
  
// Internamente hace: new  
String(clave).length
```

Operadores

Aritméticos

Operaciones matemáticas estándar:

`++, --, +, -, *, /, %`

Lógicos / Lógicos

Booleanos y cortocircuito:

`!, &&, ||`

`&&` y `||` son de cortocircuito

Comparación

Relacionales y de igualdad:

`==, !=, ===, !==, <, <=, >, >=`

Asignación

Asignación y operaciones:

`=, +=, -=, *=, /=, %=`

Manejo de Bits

`&, |, ^, ~`

32 bits, big endian y complemento a 2

Otros Operadores

`?:, typeof, +, ()`

+ (concatenación de cadenas)

Operadores

Clasificación y descripción de los tipos de operadores en JavaScript:

Categoría	Operadores	Descripción / Función
Aritméticos	<code>++, --, +, -, *, /, %</code>	Operaciones matemáticas estándar
Lógicos	<code>!, &&, </code>	Booleanos y evaluación de cortocircuito
Comparación	<code>==, !=, ===, !==, <, <=, >, >=</code>	Relacionales y de igualdad estricta
Asignación	<code>=, +=, -=, *=, /=, %=</code>	Asignación de valores y operaciones aritméticas
Manejo de Bits	<code>&, , ^, ~</code>	Operaciones a nivel de bit de 32 bits (complemento a 2)
Otros	<code>?:, typeof, +, ()</code>	Condicional, tipo, concatenación, agrupación

Actividad 5

El operador typeof

¿Para qué sirve el operador `typeof` en JavaScript?

Documentación y Recursos:

[W3Schools - JS typeof](#)

Analiza los ejemplos de la derecha y reflexiona sobre el comportamiento de los operadores aritméticos y de asignación.

¿Qué valor contiene cada variable?

Analiza la ejecución del siguiente código:

```
// Caso 1: Módulo
let anyo = 2012;
let bisiesto = anyo % 4;

// Caso 2: Pre-incremento
let variable_1 = 1;
let variable_2 = ++variable_1;

// Caso 3: Post-incremento
variable_1 = 1;
variable_2 = variable_1++;
```

Operadores

Precedencia de los operadores en JavaScript (de mayor a menor prioridad):

Prioridad	Categoría	Operadores	Descripción / Función
1	Agrupación	()	Máxima prioridad
2	Unarios	++ -- !	Incremento / lógico
3	Multiplicativos	* / %	Multiplicar, div, resto
4	Aditivos	+ -	Suma y resta
5	Relacionales	< > <= >=	Comparación
6	Igualdad	== !=	Igual y diferente
7	AND Lógico	&&	Cortocircuito
8	OR Lógico		Cortocircuito
9	Asignación	= += -= *= /= %=	Mínima prioridad (evaluado al final)

Control de flujo

¿Qué es un controlador de flujo?

Es una estructura o mecanismo en programación que determina el orden en que se ejecutan las instrucciones de un programa.

¿Por qué es esencial?

- Permite romper la ejecución puramente secuencial de arriba a abajo.
- Facilita la toma de decisiones dinámicas basadas en condiciones variables.
- Permite repetir acciones de forma automatizada mediante bucles sin duplicar código.

Bifurcación de Caminos

Gracias a las estructuras de control de flujo, el código JavaScript adquiere la capacidad de reaccionar y adaptarse de forma inteligente a:

- Entradas del usuario.
- Respuestas de servidores.
- Eventos de la interfaz gráfica.

Es la base del comportamiento dinámico en DAW.

Control de flujo

Sintaxis Condicional

```
if (expresiónDeControl) {  
    // Si la expresión es verdadera  
    ...instrucciones..  
}  
else {  
    // Si la expresión es falsa  
    ...instrucciones...  
}
```

¿Cómo funciona?

Permite bifurcar el flujo del programa en dos caminos posibles según se cumpla o no la condición:

- **Camino Verdadero (if):** Ejecuta el primer bloque de instrucciones si la expresión de control es verdadera.
- **Camino Falso (else):** Ejecuta el bloque alternativo si la expresión de control resulta ser falsa.

Control de flujo

Sintaxis Condicional

```
if (expresiónDeControl) {  
    // Si la expresión es verdadera  
    ...instrucciones...  
}  
else if (expresiónDeControl) {  
    // Si la expresión es verdadera  
    ...instrucciones...  
}  
else {  
    // Si la expresión es falsa  
    ...instrucciones...  
}
```

if ... else if ... else

Permite encadenar múltiples condiciones secuenciales cuando existen más de dos alternativas posibles:

- **Evaluación en orden:** Se comprueba cada condición secuencialmente hasta encontrar una verdadera.
- **Bloque excluyente:** Solo se ejecuta el bloque de instrucciones de la primera condición que resulte verdadera.
- **Camino por defecto (else):** El bloque final se ejecuta opcionalmente si ninguna de las condiciones anteriores se cumple.

Control de flujo

Sintaxis switch...case

```
switch (expresiónDeControl) {  
    case valor1:  
        // instrucciones si coincide con valor1  
        break;  
    case valor2:  
        // instrucciones si coincide con valor2  
        break;  
    default:  
        // si no coincide con ningún caso  
}
```

¿Cómo funciona?

Permite bifurcar el flujo del programa en múltiples ramas según el valor de la expresión de control:

- **Evaluación de casos (case):** Se compara el valor de la expresión de forma secuencial.
- **Sentencia de salida (break):** Evita la ejecución secuencial ("fall-through") saltando fuera del switch.
- **Bloque por defecto (default):** Ejecuta instrucciones opcionales si no se cumple ninguna de las opciones anteriores.

Control de flujo

Código de Ejemplo

```
let fabricante = 'citroen';
switch (fabricante) {
  case 'ford':
    alert('Talleres Velasco');
    break;
  case 'citroen':
    alert('Talleres Doctor Esquerdo');
    break; // Detiene la ejecución aquí
  default:
    alert('No existen talleres concertados');
}
```

Análisis de la Ejecución

Con la variable `fabricante = 'citroen'`, el flujo de control actúa de la siguiente manera:

- **Búsqueda de coincidencia:** Evalúa secuencialmente los casos hasta encontrar `case 'citroen'` en la línea 006.
- **Ejecución de instrucciones:** Entra en el bloque y ejecuta el `alert` correspondiente (línea 007).
- **Efecto del break:** Al llegar a la línea 008, la sentencia `break` detiene la ejecución y sale del bloque, impidiendo que se ejecuten los siguientes casos.

Control de flujo

A veces, omitir intencionadamente la sentencia **break** resulta muy útil para simplificar el código:

Código de Ejemplo

```
let edad = 8;
switch (edad) {
  case 18:
  case 17:
  case 16:
    alert('Categoría: Juvenil');
    break;
  case 15:
  case 14:
    alert('Categoría: Cadete');
    break;
  default:
    alert('Categoría: Pre-benjamín');
}
```

Análisis del "Fall-through"

Al no escribir un **break**, la ejecución del programa continúa con el siguiente caso de forma secuencial:

- **Agrupación de casos:** Las edades 18, 17 y 16 quedan englobadas en la misma alerta ('Juvenil') sin repetir código.
- **Menor redundancia:** Simplifica la estructura permitiendo múltiples condiciones de entrada para un mismo resultado.
- **Punto de salida común:** El flujo continúa cayendo en cascada hasta encontrar el primer **break** que lo detenga.

Control de flujo

while / do...while

while

```
while (expresión) {  
    // instrucciones  
}
```

do...while

Itera al menos una vez antes de evaluar la condición.

```
do {  
    // instrucciones  
} while (expresión)
```

Bucle for

Estructura ideal para realizar iteraciones con control de inicialización, condición y actualización.

```
for (inicialización;  
    expresiónControl;  
    actualización) {  
    // instrucciones  
}
```

continue y break

Sentencias para alterar el flujo de las iteraciones.

break: Abandona inmediatamente el bucle.

continue: Salta a la siguiente iteración.

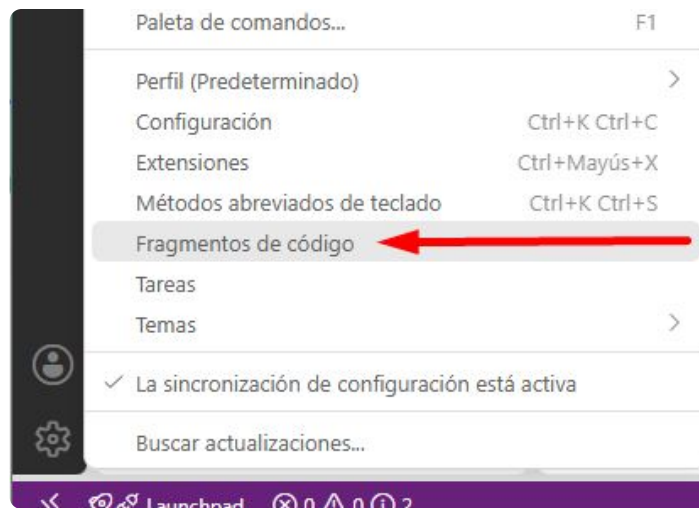
[Documentación break](#)

Actividad 6

¿Tengo que saberme de memoria la sintaxis de las estructuras de control?

- En Visual Studio Code
- En un archivo JavaScript escribe `for` y pulsa **Ctrl + Space**
- Investiga la opción **Fragmentos de código**

Fragmentos de código en VS Code



Depuración de Código

DevTools del Navegador (F12)

Dominar las herramientas de inspección es esencial en DAW para verificar la ejecución de código.

- **La Consola:** Evalúa JS en tiempo real y muestra mensajes de `console.log()`.
- **Depurador Visual (Sources):**
 - Explora la estructura de archivos del cliente.
 - **Breakpoints:** Pausan la ejecución para inspeccionar variables y memoria.
 - **debugger;**: Sentencia para forzar la pausa automática.

Guía de Depuración Práctica

Al pausar tu código en un punto de interrupción, examina la sección '**Scope**' en el panel lateral derecho:

- **Local:** Variables internas de la función activa.
- **Script/Block:** Ámbito de bloque para sentencias de tipo `let` y `const`.
- **Global:** Variables accesibles en todo el script (`window`).

 **Úsalo para resolver la Actividad 12**

Funciones

¿Qué es una función?

Como Bloque de Instrucciones

Es un conjunto de sentencias agrupadas bajo un mismo nombre. Esto permite ejecutar todo el bloque en cualquier momento del código con solo invocar dicho nombre seguido de paréntesis.

Como Objeto en JavaScript

Es un objeto de primera clase que contiene una cadena de caracteres ejecutable. Al invocar el nombre del objeto con paréntesis, este bloque de código se evalúa y ejecuta dentro del entorno de JavaScript.

Funciones

¿Qué ventajas tiene utilizar funciones?

- Reutilización del código
- Gestión/Mantenimiento del código

Evitan la duplicación de código y facilitan enormemente las tareas de depuración y escalabilidad del software.

¿Cómo se crea e invoca una función?



```
function cuadrado(x) {  
    return x * x;  
}  
  
alert(cuadrado(2));
```

Funciones: Invocación sin paréntesis

¿Qué ocurre si invoco una función sin paréntesis?

1. Error de Concepto

Omitir los paréntesis '()' no provocará la ejecución de la lógica interna de la función.

2. Referencia Directa

Estarás haciendo referencia directa al objeto de la función en sí mismo, obteniendo su definición.

3. Utilidad Avanzada

Permite asignar la función a otras variables o programar manejadores de eventos (handlers) para una ejecución diferida.

```
funcion.js

function saludar() {
  alert("¡Hola!");
}

// Caso 1: Con paréntesis (Invocación inmediata)
let resultado = saludar();
// Se ejecuta la alerta de inmediato.

// Caso 2: Sin paréntesis (Copia de referencia)
let copiaFuncion = saludar;
// No se ejecuta la alerta; copiaFuncion ahora
// apunta al mismo objeto de función.

copiaFuncion(); // Se ejecuta ahora la alerta ✓
```

Funciones

Llamadas entre funciones

Modularidad

Las funciones se pueden invocar mutuamente entre sí para resolver tareas de forma conjunta.

Orden de Declaración Libre (Hoisting)

En JavaScript tradicional, no es obligatorio mantener un orden de declaración secuencial estricto. Puedes definir una función líneas más abajo de donde fue invocada por primera vez y se ejecutará correctamente.

ejemplo.js

```
function ComprobarHumano() {  
  // Llamada a la otra función  
  let valor = prompt("Resultado de 144/12: ");  
  let res = Verificacion(Number(valor));  
  if (res) { alert(";Correcto!"); }  
  else { alert(";Incorrecto!"); }  
}  
  
// Declarada después, pero ejecutable  
function Verificacion(res) {  
  return res === 12;  
}  
  
ComprobarHumano()
```

Funciones

¿Cómo ejecutar código tras interactuar con HTML?

- **Oyentes y Manejadores (Event Handlers):** Vinculamos el código a eventos disparados por el usuario desde las etiquetas del documento HTML.
- **Manejadores en línea tradicionales:** Consiste en declarar el atributo del evento directamente en la etiqueta HTML (ej: `onclick="funcion()"`) apuntando a la función JS.
- **Nota Crítica (DAW):** Aunque `onclick` es útil didácticamente, el desarrollo profesional exige separar el contenido (HTML) del comportamiento (JS) usando `addEventListener()`.

```
index.html

<!-- Código HTML (Ejemplo didáctico) -->
<body>
  <input type="button" value="Inicia cuenta atrás"
    onclick="cuentaAtras()" />
  <script>
    function cuentaAtras() {
      let countdown = prompt("Introduce número:");
      countdown = Number(countdown);
      for (; countdown > 0; countdown--) {
        alert(countdown + "...");
      }
    }
  </script>
</body>
```

Actividad 7

Enunciado

Realiza una página HTML con un botón y un script JavaScript para que cuando el usuario pulse el botón se llame a una función que muestre por pantalla el mensaje “hola mundo”.

Requisitos clave:

- Estructura de página HTML básica
- Etiqueta de botón con manejador de evento
- Función JavaScript que use `alert()`

Actividad 7

Explicación de la Solución Estándar

- **Etiqueta <button>:** Crea el botón en el HTML con el texto 'Saludar'. El atributo **onclick** registra la función a invocar al hacer clic.
- **Bloque <script>:** Contiene la declaración explícita de la función **saludar()** encargada de llamar al alert de navegador.
- **Recuerda:** En DAW seguiremos estas plantillas de ejemplo estructuradas y añadiremos siempre comentarios explicando el flujo del script.

```
index.html

<!DOCTYPE html>
<html lang="es">
<head>
<meta charset="UTF-8">
<title>Actividad 7 - Hola Mundo</title>
</head>
<body>
<!-- Botón con manejador onclick -->
<button onclick="saludar()">Saludar</button>
<script>
// Función que muestra alerta
function saludar() {
  alert("hola mundo");
}
</script>
</body>
</html>
```

¿Dudas? ¿Preguntas?



¿Tienes alguna pregunta?

Aprovecha este momento para aclarar cualquier concepto, resolver dudas sobre las tareas o profundizar en los temas de hoy.

¡El aprendizaje no se detiene!

¡A funcionar!



Trabajo Autónomo

Completa las actividades prácticas en la plataforma educativa:

- Realizar todos los ejercicios del aula virtual **UT2**

Gracias por vuestra atención



Fin de la Presentación

¡Muchas gracias por vuestra atención y participación activa!

- Recordad revisar las tareas pendientes en el aula virtual